

Discussion sur les constructeurs de threads

Fabrice.Kordon@lip6.fr



Numérotation interne des threads

2

Passer le numéro en paramètre du constructeur

-  On contrôle la numérotation «de l'extérieur»
-  Impossible de vérifier l'unicité au niveau de la classe

Gérer l'identification en interne

-  Plus de contrôle de la numérotation
-  Mais unicité garantie

Nouvelle version de Producteur

3

```
class Producteur implements Runnable {
    static final public int cstMaxProd = 3; // constante
    private static int createurIdent = 1;
    private static final Object mutex = new Object();

    private BoiteMessage b;
    private int monNumero;
    private boolean actif = true;

    Producteur (BoiteMessage b) {
        synchronized (mutex) {
            this.monNumero = createurIdent++;
        }
        this.b = b;
    }

    private void trace (String m) {
        String indent = String.format("%" + monNumero* 2 + "s", "");
        System.out.println (indent + "Client " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }
}
```

...

Nouvelle version de la classe principale

4

```
public class ProdCons4 {  
  
    public static void main(String []args) {  
  
        BoiteMessage boite = new BoiteMessage();  
  
        Thread cons = new Thread(new Consommateur (boite));  
        Thread prods[] = new Thread [Producteur.cstMaxProd];  
  
        cons.start(); // démarrer le serveur  
  
        for (int i = 0; i < prods.length ; i++) {  
            prods[i] = new Thread(new Producteur (boite));  
            prods[i].start(); // démarrer un client  
        }  
    }  
}
```


Nouvelle version de la classe principale

4

```
public class ProdCons4 {
```

```
$ java ProdCons4
```

```
Serveur : initialisé
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
    Client 2 : initialisé
```

```
    Client 3 : initialisé
```

```
    Client 1 : initialisé
```

```
    Client 2 : terminé
```

```
    Client 3 : On recommence, Recommencer: Boîte déjà pleine
```

```
    Client 1 : On recommence, Recommencer: Boîte déjà pleine
```

```
Serveur : retire 2
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
    Client 1 : On recommence, Recommencer: Boîte déjà pleine
```

```
    Client 3 : terminé
```

```
Serveur : retire 3
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
    Client 1 : terminé
```

```
Serveur : retire 1
```

```
Serveur : terminé
```


En guise de conclusion...

5

Si on avait omis le `synchronized`?

- 🔊 Cela fonctionnerait toujours...



... mais pourquoi?

Car...

- 🔊 L'exécution du constructeur se fait en séquentiel
 - ▶ Appel par la «classe principale» seulement

Mais...

- 🔊 En général, il vaut mieux protéger
 - ▶ Qui sait comment va évoluer le programme?